Assignment 1: E-Commerce Sales Analytics

Scenario: An e-commerce company stores transactional data in MySQL and wants to move historical sales data to Hadoop for analytics.

MySQL tables: customers(customer_id, name, city, state, join_date), products(product_id, product_name, category, price), and orders(order_id, customer_id, product_id, quantity, order_date, order_status).

Perform following tasks:

1. Create the database ecommerce_db and all three tables in MySQL. Insert at least **30 customers, 15 products, and 100 orders.**

Code:

```
CREATE DATABASE ecommerce_db;

CREATE TABLE customers (
   customer_id INT PRIMARY KEY,
   name VARCHAR(100),
   city VARCHAR(50),
   state VARCHAR(50),
   join_date DATE
);


CREATE TABLE products (
   product_id INT PRIMARY KEY,
   product_name VARCHAR(100),
   category VARCHAR(50),
   price DECIMAL(10,2)
);
```

```
CREATE TABLE orders (
   order_id INT PRIMARY KEY,
   customer_id INT,
   product_id INT,
   quantity INT,
   order_date DATE,
   order_status VARCHAR(30),
   FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
   FOREIGN KEY (product_id) REFERENCES products(product_id)
);
```

2. Use Sqoop to importall three tables into separate directories under /me/ecommerce/ in HDFS.

Command:

```
sqoop import-all-tables --connect "jdbc:mysql://localhost:3306/ecommerce_db?useSSL=false" --username neel --password neel --warehouse-dir /me/ecommerce
```

3. Verify imported data using appropriate HDFS commands and report the number of files and records.
4. Create a Hive database ecommerce_dw and suitable tables over the imported data.

Code:

```
CREATE DATABASE ecommerce_dw;
```

5. Write HiveQL queries to determine:
    - Total sales revenue.
    - Revenue by product category.
    - Top 5 customers by spending.

- Top 5 products by quantity sold.
- Monthly sales revenue.
- City with the highest sales.
- Percentage of cancelled orders.

Code:

Q1)

```
INSERT OVERWRITE DIRECTORY '/meNeel/ecommerce/results/sales_by_category'
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
SELECT
   p.category,
   SUM(o.quantity * p.price) AS total_sales
FROM orders o
JOIN products p
   ON o.product_id = p.product_id
GROUP BY p.category
ORDER BY total_sales DESC;
```

Q2)

```
INSERT OVERWRITE DIRECTORY '/meNeel/ecommerce/results/top_5_products'
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
SELECT
   p.product_id,
   p.product_name,
   SUM(o.quantity * p.price) AS revenue
FROM orders o
JOIN products p
   ON o.product_id = p.product_id
GROUP BY
   p.product_id,
   p.product_name
ORDER BY revenue DESC
LIMIT 5;
```

Q3)

```
INSERT OVERWRITE DIRECTORY '/meNeel/ecommerce/results/top_5_customers'
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
SELECT
   c.customer_id,
   c.name,
```

```
    SUM(o.quantity * p.price) AS total_spending
FROM orders o
JOIN customers c
    ON o.customer_id = c.customer_id
JOIN products p
    ON o.product_id = p.product_id
GROUP BY
    c.customer_id,
    c.name
ORDER BY total_spending DESC
LIMIT 5;
```

Q4)

```
INSERT OVERWRITE DIRECTORY '/meNeel/ecommerce/results/orders_by_city'
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
SELECT
    c.city,
    COUNT(o.order_id) AS total_orders
FROM orders o
JOIN customers c
    ON o.customer_id = c.customer_id
GROUP BY c.city
ORDER BY total_orders DESC;
```

Q5)

```
INSERT OVERWRITE DIRECTORY '/meNeel/ecommerce/results/average_order_value'
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
SELECT
    AVG(o.quantity * p.price) AS average_order_value
FROM orders o
JOIN products p
    ON o.product_id = p.product_id;
```

Q6)

```
INSERT OVERWRITE DIRECTORY
'/meNeel/ecommerce/results/order_status_distribution'
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
SELECT
    order_status,
    COUNT(*) AS total_orders
FROM orders
GROUP BY order_status
ORDER BY total_orders DESC;
```

Q7)

```
INSERT OVERWRITE DIRECTORY '/meNeel/ecommerce/results/month_highest_sales'
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
SELECT
   DATE_FORMAT(order_date, 'yyyy-MM') AS order_month,
   SUM(o.quantity * p.price) AS total_sales
FROM orders o
JOIN products p
   ON o.product_id = p.product_id
GROUP BY DATE_FORMAT(order_date, 'yyyy-MM')
ORDER BY total_sales DESC
LIMIT 1;
```

6. Store the result of the individual query into separate file in HDFS .
7. Transfer back these result files to local system.

Command:

1. hdfs dfs -get /me/ecommerce/results/ ~/ecommerce_results/

Assignment 2: University Student Performance Analytics

Scenario: A university maintains student, course, enrollment, and examination information in MySQL. The university wants to analyze academic performance using Hadoop.

MySQL tables: students(student_id, student_name, department, admission_year, gender), courses(course_id, course_name, department, credits), and results(result_id, student_id, course_id, semester, marks, attendance).

Perform following tasks:

1. Create university_db in MySQL and populate it with at least **40 students, 10 courses, and 150 result records.**

Code:

```
CREATE DATABASE IF NOT EXISTS university_db;

USE university_db;
```

2. Import the tables using Sqoop into /me/university/.

Command:

```
hdfs dfs -mkdir -p /me/university
```

3. Import results using **multiple Sqoop mappers** and explain the purpose of --split-by.

Code:

1. sqoop import --connect jdbc:mysql://localhost:3306/university_db --username neel --password 'neel' --table students --target-dir /me/university/students --fields-terminated-by ',' – m 1

2. sqoop import --connect jdbc:mysql://localhost:3306/university_db --username neel --password 'neel' --table courses --target-dir /me/university/courses --fields-terminated-by ',' – m 1

3. sqoop import --connect jdbc:mysql://localhost:3306/university_db --username neel --password 'neel' --table results --target-dir /me/university/results --fields-terminated-by ',' – m 4

Explanation:

--split-by tells Sqoop which column to use to divide the table's rows among the parallel mappers you asked for (-m 4 = 4 mappers).

Sqoop finds the min and max of that column, splits the range into roughly equal chunks, and each mapper pulls its own chunk at the same time that's what actually makes the import faster.

You usually pick a column like the primary key (e.g. result_id) because it's numeric and evenly spread, so no single mapper gets stuck doing way more work than the others.

4. Inspect the resulting HDFS directories and files.

- Directory tree

  hdfs dfs -ls -R /me/university

  hdfs dfs -ls /me/university/students

  hdfs dfs -ls /me/university/courses

  hdfs dfs -ls /me/university/results

- Peek at content

  hdfs dfs -cat /me/university/students/part-m-00000 | head -10

  hdfs dfs -cat /me/university/results/part-m-00000 | head -10

- File sizes / block info:

  hdfs dfs -du -h /me/university

  hdfs fsck /me/university/results -files -blocks -locations

5. Create appropriate Hive tables.

Code:

1.

```
CREATE DATABASE IF NOT EXISTS university;

USE university;
```

2.

```
CREATE EXTERNAL TABLE students (
   student_id      INT,
   student_name    STRING,
   department      STRING,
   admission_year  INT,
   gender          STRING
)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
STORED AS TEXTFILE
LOCATION '/me/university/students';
```

3.

```
CREATE EXTERNAL TABLE courses (
   course_id       INT,
   course_name     STRING,
   department      STRING,
   credits         INT
)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
STORED AS TEXTFILE
LOCATION '/me/university/courses';
```

4.

```
CREATE EXTERNAL TABLE results (
   result_id       INT,
   student_id      INT,
   course_id       INT,
   semester        INT,
```

```
    marks          INT,
    attendance     INT
)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
STORED AS TEXTFILE
LOCATION '/me/university/results';
```

6. Write Hive queries to find:
    - Average marks for each department.
    - Top 3 students in each department.
    - Pass percentage for each course.
    - Students having attendance below 75%.
    - Relationship between attendance and average marks.
    - Course having the highest failure rate.
    - Semester-wise average performance.

Code:

1.
```
SELECT s.department, ROUND(AVG(r.marks),2) AS avg_marks
FROM results r
JOIN students s ON r.student_id = s.student_id
GROUP BY s.department
ORDER BY avg_marks DESC;
```

2.
```
SELECT department, student_id, student_name, avg_marks, rnk
FROM (
   SELECT s.department, s.student_id, s.student_name,
        ROUND(AVG(r.marks),2) AS avg_marks,
        RANK() OVER (PARTITION BY s.department ORDER BY AVG(r.marks) DESC)
AS rnk
   FROM results r
   JOIN students s ON r.student_id = s.student_id
   GROUP BY s.department, s.student_id, s.student_name
) ranked
WHERE rnk <= 3
ORDER BY department, rnk;
```

3. SELECT c.course_id, c.course_name,

   ROUND(100.0 * SUM(CASE WHEN r.marks >= 40 THEN 1 ELSE 0 END) / COUNT(*), 2) AS pass_pct

   FROM results r

   JOIN courses c ON r.course_id = c.course_id

   GROUP BY c.course_id, c.course_name

   ORDER BY pass_pct DESC;

4. SELECT DISTINCT s.student_id, s.student_name, s.department, r.attendance
   FROM results r

   JOIN students s ON r.student_id = s.student_id

   WHERE r.attendance < 75

   ORDER BY r.attendance ASC;

5. SELECT

   CASE

   WHEN attendance < 60 THEN '<60%'

   WHEN attendance BETWEEN 60 AND 74 THEN '60-74%'

   WHEN attendance BETWEEN 75 AND 89 THEN '75-89%'

   ELSE '90-100%'

   END AS attendance_band,

   ROUND(AVG(marks),2) AS avg_marks,

   COUNT(*) AS num_records

   FROM results

   GROUP BY

   CASE

   WHEN attendance < 60 THEN '<60%'

   WHEN attendance BETWEEN 60 AND 74 THEN '60-74%'

   WHEN attendance BETWEEN 75 AND 89 THEN '75-89%'

   ELSE '90-100%'

   END

   ORDER BY attendance_band;

6. SELECT c.course_id, c.course_name,
   ROUND(100.0 * SUM(CASE WHEN r.marks < 40 THEN 1 ELSE 0 END) / COUNT(*), 2) AS failure_pct
   FROM results r
   JOIN courses c ON r.course_id = c.course_id
   GROUP BY c.course_id, c.course_name
   ORDER BY failure_pct DESC
   LIMIT 1;

7. SELECT semester, ROUND(AVG(marks),2) AS avg_marks, COUNT(*) AS num_results
   FROM results
   GROUP BY semester
   ORDER BY semester;

7. Store the result of the individual query into separate file in HDFS .

1.)

```
INSERT OVERWRITE DIRECTORY '/meNeel/university/output/Que1'

ROW FORMAT DELIMITED FIELDS TERMINATED BY ','

SELECT s.department, ROUND(AVG(r.marks),2)

FROM results r JOIN students s ON r.student_id = s.student_id

GROUP BY s.department;
```

2.)

```
INSERT OVERWRITE DIRECTORY '/meNeel/university/output/Que2'

ROW FORMAT DELIMITED FIELDS TERMINATED BY ','

SELECT department, student_id, student_name, avg_marks, rnk

FROM (

   SELECT s.department, s.student_id, s.student_name,

        ROUND(AVG(r.marks),2) AS avg_marks,

        RANK() OVER (PARTITION BY s.department ORDER BY AVG(r.marks) DESC) AS
rnk

   FROM results r JOIN students s ON r.student_id = s.student_id

   GROUP BY s.department, s.student_id, s.student_name

) ranked

WHERE rnk <= 3;
```

3.)

```
INSERT OVERWRITE DIRECTORY '/meNeel/university/output/Que3'
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
SELECT c.course_id, c.course_name,
      ROUND(100.0 * SUM(CASE WHEN r.marks >= 40 THEN 1 ELSE 0 END) / COUNT(*),
2)
FROM results r JOIN courses c ON r.course_id = c.course_id
GROUP BY c.course_id, c.course_name;
```

4.)

```
INSERT OVERWRITE DIRECTORY '/meNeel/university/output/Que4'
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
SELECT DISTINCT s.student_id, s.student_name, s.department, r.attendance
FROM results r JOIN students s ON r.student_id = s.student_id
WHERE r.attendance < 75;
```

5.)

```
INSERT OVERWRITE DIRECTORY '/meNeel/university/output/Que5'
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
SELECT
  CASE
    WHEN attendance < 60 THEN '<60%'
    WHEN attendance BETWEEN 60 AND 74 THEN '60-74%'
    WHEN attendance BETWEEN 75 AND 89 THEN '75-89%'
    ELSE '90-100%'
  END AS band,
  ROUND(AVG(marks),2), COUNT(*)
FROM results
GROUP BY
  CASE
```

```
    WHEN attendance < 60 THEN '<60%'
    WHEN attendance BETWEEN 60 AND 74 THEN '60-74%'
    WHEN attendance BETWEEN 75 AND 89 THEN '75-89%'
    ELSE '90-100%'
  END;
```

6.)

```
INSERT OVERWRITE DIRECTORY '/meNeel/university/output/Que6'
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
SELECT c.course_id, c.course_name,
       ROUND(100.0 * SUM(CASE WHEN r.marks < 40 THEN 1 ELSE 0 END) / COUNT(*), 2)
AS failure_pct
FROM results r JOIN courses c ON r.course_id = c.course_id
GROUP BY c.course_id, c.course_name
ORDER BY failure_pct DESC
LIMIT 1;
```

7.)

```
INSERT OVERWRITE DIRECTORY '/meNeel/university/output/Que7'
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
SELECT semester, ROUND(AVG(marks),2), COUNT(*)
FROM results
GROUP BY semester;
```

8. Transfer back these result files to local system.

Command: hdfs dfs -get /me/university/results/ ~/university_results/

Assignment 3: Banking Transaction and Customer Analytics

Scenario: A bank stores customer accounts and transactions in MySQL. Its analytics team needs a Hadoop-based solution for transaction analysis.

MySQL tables: customers(customer_id, customer_name, city, account_type, account_open_date), accounts(account_id, customer_id, balance, branch), and transactions(transaction_id, account_id, transaction_type, amount, transaction_date).

Populate the database with at least **30 customers, 40 accounts, and 200 transactions**.

Tasks:

1. Create and populate bank_db.

Command: DROP DATABASE IF EXISTS bank_db;
CREATE DATABASE bank_db;
USE bank_db;

2. Perform a **full Sqoop import** of customers, accounts and transactions.

Code:

1.

```
sqoop import \

--connect jdbc:mysql://localhost:3306/bank_db \

--username sqoop \

--password neel \

--table customers \

--target-dir /user/sois/bank_db/customers \

--m 1
```

2.Import accounts:

```
sqoop import \

--connect jdbc:mysql://localhost:3306/bank_db \

--username neel \

--password neel \

--table accounts \

--target-dir /user/sois/bank_db/accounts --m 1
```

3.Import transactions:

```
sqoop import \
--connect jdbc:mysql://localhost:3306/bank_db \
--username neel \
--password neel \
--table transactions \
--target-dir /user/sois/bank_db/transactions \
--m 1
```

3. Verify through HDFS that the new data has been imported.

Commands:

```
hdfs dfs -ls /user/sois/bank_db
hdfs dfs -ls /user/sois/bank_db/customers
hdfs dfs -ls /user/sois/bank_db/accounts
hdfs dfs -ls /user/sois/bank_db/transactions
```

4. Create Hive tables and answer:
    - Total deposits and withdrawals.
    - Customers with the highest transaction values.
    - Branch-wise account balances.
    - Average transaction amount by account type.
    - Month-wise transaction volume.
    - Accounts with no transactions.
    - Customers whose withdrawal amount exceeds their deposit amount.
5. Store the result of the individual query into separate file in HDFS .

Code:

1.
```
SELECT
   transaction_type,
   SUM(amount) AS total_amount
FROM transactions
GROUP BY transaction_type;

INSERT OVERWRITE DIRECTORY '/user/sois/bank_results/total_deposits_withdrawals'
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','< Same Query Again >
```

2.)
```
SELECT
    c.customer_id,
    c.customer_name,
    SUM(t.amount) AS total_transaction_value
FROM customers c
JOIN accounts a
    ON c.customer_id = a.customer_id
JOIN transactions t
    ON a.account_id = t.account_id
GROUP BY c.customer_id, c.customer_name
ORDER BY total_transaction_value DESC;

INSERT OVERWRITE DIRECTORY
'/user/sois/bank_results/highest_transaction_customers'
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','< Same Query Again >
```

3.)
```
SELECT
    branch,
    SUM(balance) AS total_balance
FROM accounts
GROUP BY branch
ORDER BY total_balance DESC;

INSERT OVERWRITE DIRECTORY '/user/sois/bank_results/branch_balances'
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','< Same Query Again >
```

4.)
```
SELECT
    c.account_type,
    AVG(t.amount) AS average_transaction_amount
FROM customers c
JOIN accounts a
    ON c.customer_id = a.customer_id
JOIN transactions t
    ON a.account_id = t.account_id
GROUP BY c.account_type;

INSERT OVERWRITE DIRECTORY
'/user/sois/bank_results/avg_transaction_account_type'
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','< Same Query Again >
```

5.)
```
SELECT
    YEAR(transaction_date) AS year,
    MONTH(transaction_date) AS month,
    COUNT(*) AS transaction_volume
FROM transactions
GROUP BY YEAR(transaction_date), MONTH(transaction_date)
ORDER BY year, month;
INSERT OVERWRITE DIRECTORY '/user/sois/bank_results/monthly_transaction_volume'
```

```
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ',' < Same Query Again >
```

6.)
```
SELECT
    a.account_id,
    a.customer_id,
    a.balance,
    a.branch
FROM accounts a
LEFT JOIN transactions t
    ON a.account_id = t.account_id
WHERE t.transaction_id IS NULL;

INSERT OVERWRITE DIRECTORY '/user/sois/bank_results/accounts_no_transactions'
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ',' < Same Query Again >
```

7.)
```
SELECT
    c.customer_id,
    c.customer_name,
    SUM(CASE
        WHEN t.transaction_type = 'Withdrawal'
        THEN t.amount ELSE 0
    END) AS total_withdrawal,
    SUM(CASE
        WHEN t.transaction_type = 'Deposit'
        THEN t.amount ELSE 0
    END) AS total_deposit
FROM customers c
JOIN accounts a
    ON c.customer_id = a.customer_id
JOIN transactions t
    ON a.account_id = t.account_id
GROUP BY c.customer_id, c.customer_name
HAVING
    SUM(CASE
        WHEN t.transaction_type = 'Withdrawal'
        THEN t.amount ELSE 0
    END)
    >
    SUM(CASE
        WHEN t.transaction_type = 'Deposit'
        THEN t.amount ELSE 0
    END);
INSERT OVERWRITE DIRECTORY '/user/sois/bank_results/withdrawal_exceeds_deposit'
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ',' < Same Query Again >
```

6. Transfer back these result files to local system.

Command: hdfs dfs -get /me/customer/results/ ~/customer_results/

Assignment 4: Hospital Operations Analytics

Scenario: A multi-specialty hospital wants to analyze patient visits, doctors, departments, and billing information stored in MySQL.

MySQL tables: patients(patient_id, patient_name, city, age, gender), doctors(doctor_id, doctor_name, specialization, department), visits(visit_id, patient_id, doctor_id, visit_date, diagnosis, consultation_fee), and payments(payment_id, visit_id, payment_mode, amount, payment_status).

Students should create at least **40 patients, 12 doctors, 150 visits, and 150 payment records** using synthetic data only.

Tasks:

1. Create and populate hospital_db.

Command: CREATE DATABASE hospital_db;

USE hospital_db;

2. Import the four tables using Sqoop.

Command: sqoop import-all-tables \

--connect "jdbc:mysql://localhost:3306/hospital_db?useSSL=false" \

--username neel \

--password neel \

--warehouse-dir /me/hospital \

--m 1

3. Store them under a structured HDFS hierarchy such as /me/hospital/patients, /doctors, /visits, and /payments.
4. Create Hive tables.

Code: 1.)

Create patients table and load the data in HDFS

```
CREATE TABLE patients (
  patient_id INT,
  patient_name STRING,
  city STRING,
  age INT,
```

```
    gender STRING
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ',';

LOAD DATA INPATH '/me/hospital/patients'
INTO TABLE patients;
```

2.)

```
CREATE TABLE doctors (
    doctor_id INT,
    doctor_name STRING,
    specialization STRING,
    department STRING
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ',';

LOAD DATA INPATH '/me/hospital/doctors'
INTO TABLE doctors;
```

3.)

```
CREATE TABLE visits (
    visit_id INT,
    patient_id INT,
    doctor_id INT,
    visit_date DATE,
    diagnosis STRING,
    consultation_fee DECIMAL(10,2)
)
ROW FORMAT DELIMITED
```

```
FIELDS TERMINATED BY ',';

LOAD DATA INPATH '/me/hospital/visits'
INTO TABLE visits;
```

4.)

```
CREATE TABLE payments (
   payment_id INT,
   visit_id INT,
   payment_mode STRING,
   amount DECIMAL(10,2),
   payment_status STRING
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ',';
LOAD DATA INPATH '/me/hospital/payments'
INTO TABLE payments;
```

5. Use Hive joins and aggregations to determine:
    - Number of visits per department.
    - Revenue generated by each department.
    - Top 5 doctors based on number of consultations.
    - Average consultation fee by specialization.
    - Most common diagnoses.
    - City-wise patient counts.
    - Percentage of successful versus pending payments.
    - Month having the highest number of visits.

6. Store the result of the individual query into separate file in HDFS .

Code:

Q1)

```
INSERT OVERWRITE DIRECTORY '/me/hospital/results/visits_per_department'
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
SELECT
   d.department,
   COUNT(v.visit_id) AS total_visits
FROM visits v
JOIN doctors d
   ON v.doctor_id = d.doctor_id
GROUP BY d.department
ORDER BY total_visits DESC;
```

Q2)

```
INSERT OVERWRITE DIRECTORY '/me/hospital/results/revenue_per_department'
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
SELECT
   d.department,
   SUM(v.consultation_fee) AS total_revenue
FROM visits v
JOIN doctors d
   ON v.doctor_id = d.doctor_id
GROUP BY d.department
ORDER BY total_revenue DESC;
```

Q3)

```
INSERT OVERWRITE DIRECTORY '/me/hospital/results/top_5_doctors'
ROW FORMAT DELIMITED
```

```
FIELDS TERMINATED BY ','
SELECT
    d.doctor_id,
    d.doctor_name,
    d.specialization,
    COUNT(v.visit_id) AS total_consultations
FROM doctors d
JOIN visits v
    ON d.doctor_id = v.doctor_id
GROUP BY
    d.doctor_id,
    d.doctor_name,
    d.specialization
ORDER BY total_consultations DESC
LIMIT 5;
```

Q4)

```
INSERT OVERWRITE DIRECTORY '/me/hospital/results/avg_fee_specialization'
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
SELECT
    d.specialization,
    AVG(v.consultation_fee) AS average_consultation_fee
FROM doctors d
JOIN visits v
    ON d.doctor_id = v.doctor_id
GROUP BY d.specialization
ORDER BY average_consultation_fee DESC;
```

Q5)

```
INSERT OVERWRITE DIRECTORY '/me/hospital/results/most_common_diagnoses'
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
SELECT
   diagnosis,
   COUNT(*) AS total_cases
FROM visits
GROUP BY diagnosis
ORDER BY total_cases DESC;
```

Q6)

```
INSERT OVERWRITE DIRECTORY '/me/hospital/results/city_wise_patients'
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
SELECT
   city,
   COUNT(*) AS patient_count
FROM patients
GROUP BY city
ORDER BY patient_count DESC;
```

Q7)

```
INSERT OVERWRITE DIRECTORY '/me/hospital/results/payment_status_percentage'
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
SELECT
   payment_status,
   COUNT(*) AS total_payments,
```

```
   ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM payments), 2) AS percentage
FROM payments
GROUP BY payment_status;
```

Q8)

```
INSERT OVERWRITE DIRECTORY '/me/hospital/results/month_highest_visits'
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
SELECT
   DATE_FORMAT(visit_date, 'yyyy-MM') AS visit_month,
   COUNT(*) AS total_visits
FROM visits
GROUP BY DATE_FORMAT(visit_date, 'yyyy-MM')
ORDER BY total_visits DESC
LIMIT 1;
```

7. Transfer back these result files to local system.

Command: hdfs dfs -get /me/hospital/results/ ~/hospital_results/

Assignment 5: Retail Data Warehouse — Complete ETL Challenge

Scenario: A retail chain has multiple stores. Operational information is maintained in MySQL, while management wants a Hadoop/Hive analytical warehouse. This assignment combines the concepts from the previous four and is the most advanced.

MySQL tables: stores(store_id, store_name, city, state), customers(customer_id, customer_name, city, customer_segment), products(product_id, product_name, category, unit_price), and sales(sale_id, store_id, customer_id, product_id, quantity, discount, sale_date).

Create at least **5 stores, 50 customers, 20 products, and 250 sales transactions**.

Tasks:

1. Design and create retail_data in MySQL with appropriate primary and foreign keys.

Command:

```
Create MySQL Tables
Stores
CREATE TABLE stores (
   store_id INT PRIMARY KEY,
   store_name VARCHAR(100),
   city VARCHAR(50),
   state VARCHAR(50)
);
Customers
CREATE TABLE customers (
   customer_id INT PRIMARY KEY,
   customer_name VARCHAR(100),
   city VARCHAR(50),
   customer_segment VARCHAR(50)
);
Products
CREATE TABLE products (
   product_id INT PRIMARY KEY,
   product_name VARCHAR(100),
   category VARCHAR(50),
   unit_price DECIMAL(10,2)
);
Sales
CREATE TABLE sales (
   sale_id INT PRIMARY KEY,
   store_id INT,
   customer_id INT,
   product_id INT,
   quantity INT,
   discount DECIMAL(5,2),
   sale_date DATE,
   FOREIGN KEY (store_id) REFERENCES stores(store_id),
   FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
```

```
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);
```

2. Populate realistic synthetic data, including some NULL values and discounted transactions.
3. Import dimension tables using Sqoop.

Command:

```
sqoop import-all-tables \
--connect "jdbc:mysql://localhost:3306/retail_data?useSSL=false" \
--username neel \
--password neel \
--warehouse-dir /me/retail \
--m 1
```

4. Store the imported datasets in an organized HDFS directory structure.

- Load HDFS Data into Hive:

```
LOAD DATA INPATH '/me/retail/stores' INTO TABLE stores;
LOAD DATA INPATH '/me/retail/customers' INTO TABLE customers;
LOAD DATA INPATH '/me/retail/products' INTO TABLE products;
LOAD DATA INPATH '/me/retail/sales' INTO TABLE sales_raw;
```

5. Create a Hive database retail_dw and tables.

Command:

```
CREATE DATABASE retail_dw;
USE retail_dw;

CREATE TABLE stores (
    store_id INT,
    store_name STRING,
    city STRING,
    state STRING
)
```

```
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ',';
```

Customers

```
CREATE TABLE customers (
    customer_id INT,
    customer_name STRING,
    city STRING,
    customer_segment STRING
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ',';
```

Products

```
CREATE TABLE products (
    product_id INT,
    product_name STRING,
    category STRING,
    unit_price DECIMAL(10,2)
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ',';
```

Raw Sales

```
CREATE TABLE sales_raw (
    sale_id INT,
    store_id INT,
    customer_id INT,
    product_id INT,
    quantity INT,
    discount DECIMAL(5,2),
    sale_date DATE
```

```
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ',';
```

6. Perform necessary data cleaning/transformation in Hive, including handling NULLs and calculating:

   gross_amount = quantity × unit_price
   net_amount = gross_amount × (1 - discount/100)

Command:

```
CREATE TABLE sales_cleaned AS
SELECT
  s.sale_id,
  s.store_id,
  s.customer_id,
  s.product_id,
  s.quantity,
  COALESCE(s.discount, 0) AS discount,
  s.sale_date,
  p.unit_price,
  (s.quantity * p.unit_price) AS gross_amount,
  (s.quantity * p.unit_price) *
    (1 - COALESCE(s.discount, 0) / 100) AS net_amount
FROM sales_raw s
JOIN products p ON s.product_id = p.product_id;
```

8. Write analytical queries for:
    - Store-wise revenue.
    - State-wise revenue.
    - Category-wise revenue.
    - Top 10 customers.
    - Top 5 products.
    - Monthly sales trends.
    - Average order value by customer segment.
    - Store contributing the highest percentage of total revenue.
    - Products never sold.
    - Month-over-month revenue growth.

Code:

```
Q1)
SELECT
   st.store_id,
   st.store_name,
   SUM(sc.net_amount) AS total_revenue
FROM sales_cleaned sc
JOIN stores st
ON sc.store_id = st.store_id
GROUP BY st.store_id, st.store_name
ORDER BY total_revenue DESC;

Q2)
SELECT
   st.state,
   SUM(sc.net_amount) AS total_revenue
FROM sales_cleaned sc
JOIN stores st
ON sc.store_id = st.store_id
GROUP BY st.state
ORDER BY total_revenue DESC;

Q3)
SELECT
   p.category,
   SUM(sc.net_amount) AS total_revenue
FROM sales_cleaned sc
JOIN products p
ON sc.product_id = p.product_id
GROUP BY p.category
ORDER BY total_revenue DESC;
```

Q4)

```
SELECT
   c.customer_id,
   c.customer_name,
   SUM(sc.net_amount) AS total_spending
FROM sales_cleaned sc
JOIN customers c
ON sc.customer_id = c.customer_id
GROUP BY c.customer_id, c.customer_name
ORDER BY total_spending DESC
LIMIT 10;
```

Q5)

```
SELECT
   p.product_id,
   p.product_name,
   SUM(sc.net_amount) AS total_revenue
FROM sales_cleaned sc
JOIN products p
ON sc.product_id = p.product_id
GROUP BY p.product_id, p.product_name
ORDER BY total_revenue DESC
LIMIT 5;
```

Q6)

```
SELECT
   YEAR(sale_date) AS year,
   MONTH(sale_date) AS month,
   SUM(net_amount) AS total_revenue
FROM sales_cleaned
GROUP BY YEAR(sale_date), MONTH(sale_date)
ORDER BY year, month;
```

Q7)

```
SELECT
   COALESCE(c.customer_segment, 'Unknown') AS customer_segment,
   AVG(sc.net_amount) AS average_order_value
FROM sales_cleaned sc
JOIN customers c
ON sc.customer_id = c.customer_id
GROUP BY COALESCE(c.customer_segment, 'Unknown')
ORDER BY average_order_value DESC;
```

Q8)

```
SELECT
   st.store_id,
   st.store_name,
   SUM(sc.net_amount) AS store_revenue,
   (SUM(sc.net_amount) / total.total_revenue) * 100
      AS revenue_percentage
FROM sales_cleaned sc
```

```
JOIN stores st
ON sc.store_id = st.store_id
CROSS JOIN (
    SELECT SUM(net_amount) AS total_revenue
    FROM sales_cleaned
) total
GROUP BY st.store_id, st.store_name, total.total_revenue
ORDER BY revenue_percentage DESC
LIMIT 1;

Q9)
SELECT
    p.product_id,
    p.product_name,
    p.category,
    p.unit_price
FROM products p
LEFT JOIN sales_cleaned sc
ON p.product_id = sc.product_id
WHERE sc.product_id IS NULL
ORDER BY p.product_id;

Q10)
WITH monthly_revenue AS (
    SELECT
        YEAR(sale_date) AS year,
        MONTH(sale_date) AS month,
        SUM(net_amount) AS revenue
    FROM sales_cleaned
    GROUP BY YEAR(sale_date), MONTH(sale_date)
),
revenue_with_previous AS (
    SELECT
        year,
        month,
        revenue,
        LAG(revenue) OVER (ORDER BY year, month) AS previous_revenue
    FROM monthly_revenue
)
SELECT
    year,
    month,
    revenue,
    previous_revenue,
    ((revenue - previous_revenue) / previous_revenue) * 100
        AS growth_percentage
FROM revenue_with_previous
ORDER BY year, month;
```